Experiment-8

Aim: Demonstrate the Readers writers Problem.

Description

The readers-writers algorithm coordinates concurrent access to a shared resource, allowing multiple readers to read simultaneously while forcing writers to have exclusive access.

Algorithm

I. Global Initialization Step 1: Initialize semaphore mutex to 1 (protects the read_count variable). Step 2: Initialize semaphore writing to 1 (protects the shared data). Step 3: Initialize integer read_count to 0. Step 4: Initialize shared integer data to 0.

II. Algorithm for Reader Process Step 1: Start. Step 2: Wait on mutex semaphore [wait(mutex)] to ensure mutual exclusion for read_count. Step 3: Increment read_count by 1. Step 4: Check if read_count == 1 (First reader). If true, wait on writing semaphore [wait(writing)] to block writers. Step 5: Signal mutex semaphore [signal(mutex)] to allow other readers to enter. Step 6: Perform the READ operation (print shared data). Step 7: Wait on mutex semaphore [wait(mutex)] to safely update read_count. Step 8: Decrement read_count by 1. Step 9: Check if read_count == 0 (Last reader). If true, signal writing semaphore [signal(writing)] to unblock writers. Step 10: Signal mutex semaphore [signal(mutex)]. Step 11: Stop.

III. Algorithm for Writer Process Step 1: Start. Step 2: Wait on writing semaphore [wait(writing)] to acquire exclusive lock on shared data. Step 3: Perform the WRITE operation (modify shared data). Step 4: Signal writing semaphore [signal(writing)] to release the lock. Step 5: Stop.

IV. Main Execution Flow Step 1: Start. Step 2: Read the total number of reader threads and writer threads to create. Step 3: Initialize the mutex and writing semaphores. Step 4: Create the specified number of Reader threads using pthread_create(). Step 5: Create the specified number of Writer threads using pthread_create(). Step 6: Wait for all Writer threads to complete execution using pthread_join(). Step 7: Wait for all Reader threads to complete execution using pthread_join(). Step 8: Destroy both semaphores using sem_destroy(). Step 9: Stop.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
sem_t writing;
sem_t mutex;
int read_count = 0;
int data = 0;
void *reader(void *arg){
int id = *(int *)arg;
    sem_wait(&mutex);
    read_count++;
    if (read_count == 1)
        sem_wait(&writing);
    sem_post(&mutex);
    printf("Reader %d is reading data = %d\n", id, data);
    sleep(1);
    sem_wait(&mutex);
    read_count--;
    if (read_count == 0)
        sem_post(&writing);
    sem_post(&mutex);
```

```
    return NULL;
}
void *writer(void *arg){
int id = *(int *)arg;
    sem_wait(&writing);
    data++;
    printf("Writer %d is writing data = %d\n", id, data);
    sleep(1);
    sem_post(&writing);
    return NULL;
}
int main(){
int i,j;
printf("Enter the number of readers and writers :");
scanf("%d %d", &i,&j);
pthread_t r[i];
pthread_t w[j];
int rid[i], wid[j];
    sem_init(&writing, 0, 1);
    sem_init(&mutex, 0, 1);
    for (int a = 0; a < i; a++){
        rid[a] = a + 1;
        pthread_create(&r[a], NULL, reader, &rid[a]);
    }
    for (int a = 0; a < j; a++){
        wid[a] = a + 1;
        pthread_create(&w[a], NULL, writer, &wid[a]);
    }
    for (int a = 0; a < j; a++)
        pthread_join(w[a], NULL);
    for (int a = 0; a < i; a++)
        pthread_join(r[a], NULL);
    sem_destroy(&writing);
    sem_destroy(&mutex);
    return 0;
}
```

Output:

